Mini Trello: Feature Spec and Build Plan

A task and project manager with JWT auth (access + refresh tokens), a drag-and-drop kanban board, and per-board roles (owner or member). Assumed stack: React, Express, MongoDB with Mongoose, since that is what you are already learning.

1. Core concepts

- **Board** = a project. Has an owner and zero or more members.
- **List** = a column on a board (To Do, Doing, Done, or anything the owner names).
- **Card** = a task inside a list. Cards can be reordered inside a list and moved between lists.
- **Role** is per board, not global. The same user can be owner of one board and member of another.

2. Roles and permissions

Action	Owner	Member
View board, lists, cards	Yes	Yes
Create / edit / move / delete cards	Yes	Yes
Assign a card, set due date	Yes	Yes
Create / rename / reorder / delete lists	Yes	No
Rename board, edit description	Yes	No
Invite or remove members	Yes	No
Delete board	Yes	No
Leave board	No (must transfer or delete)	Yes

Decision to make early: should members be allowed to delete cards created by others? The table above says yes for simplicity. Tighten it later if you want.

3. Feature list

MVP (build these first)

Auth

- Register with name, email, password
- Login, logout, logout from all devices
- Short-lived access token, long-lived refresh token with rotation
- "Who am I" endpoint

Boards

- Create, list my boards (owned + shared), view one, rename, delete
- Invite an existing user by email as a member
- Remove a member, member can leave

Lists

- Create, rename, delete, reorder

Cards

- Create, view, edit (title, description, due date, priority, assignee), delete
- Move between lists and reorder, persisted

Frontend

- Login and register pages, protected routes
- Boards dashboard
- Board page with draggable cards
- UI hides actions the user's role cannot perform (server still enforces)

Stretch (only after MVP works end to end)

- Drag lists to reorder
- Labels with colors, card filtering
- Comments on cards
- Activity log per board
- Transfer ownership
- Pending invites for emails that have no account yet

- Search cards on a board

4. Data models

Field lists only. You write the Mongoose schemas yourself.

User

- name: string, required
- email: string, required, unique, stored lowercase and trimmed
- passwordHash: string, required (never store or return the plain password; also never return the hash in responses)
- createdAt / updatedAt: timestamps

RefreshToken (one document per login session)

- user: reference to User, indexed
- tokenHash: string (store a hash of the token, not the token itself)
- expiresAt: date (add a TTL index so MongoDB cleans up expired ones)
- revokedAt: date or null
- replacedBy: reference to the next token in the chain, or null (used to detect reuse)
- userAgent: string, optional (helps you show "active sessions" later)
- createdAt: timestamp

Board

- title: string, required, max length of your choice
- description: string, optional
- owner: reference to User, required
- members: array of objects, each with user (reference) and role ("owner" or "member"). The owner is also in this array with role owner, so every permission check is one lookup.
- timestamps

Why members are embedded: a board has a small, bounded number of members, and you read them on every request. Embedding avoids a second query. Look up the MongoDB docs section on "embedded vs referenced data" in the data modeling guide to understand

the trade-off.

List

- board: reference to Board, required, indexed
- title: string, required
- position: number, required
- timestamps

Card

- board: reference to Board (denormalized so permission checks do not need to go through the list)
- list: reference to List, required, indexed
- title: string, required
- description: string, optional
- position: number, required
- assignee: reference to User or null (must be a member of the board)
- dueDate: date or null
- priority: one of low, medium, high, or null
- createdBy: reference to User
- timestamps

Index idea: lists by (board, position) and cards by (list, position).

Ordering strategy (important design decision)

Storing positions as 0,1,2,3 means every move rewrites many documents. A common alternative is **fractional positions**: a card dropped between positions 1000 and 2000 gets 1500. Only the moved card is updated. Occasionally the gaps get too small, so you rebalance a list (renumber 1000, 2000, 3000...). Search for "fractional indexing" to read about it. Recommended for this project.

5. Auth design

- **Access token**: JWT, around 15 minutes, sent in the Authorization header as a Bearer token. Payload holds only the user id (and maybe email). On the React side keep it in memory (state/context), not localStorage.

- **Refresh token**: random opaque string or JWT, around 7 days, sent only as an httpOnly cookie with SameSite set and Secure in production. Stored hashed in the database.
- **Rotation**: every call to refresh issues a new access token and a new refresh token, and marks the old refresh token as replaced.
- **Reuse detection**: if a token that is already replaced or revoked gets presented, assume theft and revoke every session for that user.
- **Logout**: revoke the current refresh token and clear the cookie. Logout-all revokes every token for the user.
- **Passwords**: hash with bcrypt or argon2. Enforce a minimum length. Return the same generic error for "wrong email" and "wrong password".

Docs to read: OWASP Cheat Sheet Series (JSON Web Token, Password Storage, Session Management), the jwt.io introduction, the Express docs on `res.cookie` and the `cors` package's `credentials` option, and MDN's page on the `Set-Cookie` header.

6. API routes

All routes are prefixed with /api. "Auth" means a valid access token is required. Board-scoped routes also need the user to be a member of that board.

Auth

Method	Path	Auth	Purpose
POST	/auth/register	No	Create account
POST	/auth/login	No	Returns access token, sets refresh cookie
POST	/auth/refresh	Cookie	Rotates refresh token, returns new access token
POST	/auth/logout	Cookie	Revokes current session
POST	/auth/logout-all	Yes	Revokes all sessions
GET	/users/me	Yes	Current user profile

Boards

Method	Path	Role	Purpose
GET	/boards	any user	Boards I own or belong to
POST	/boards	any user	Create board (caller becomes owner)
GET	/boards/:boardId	member+	Board with its lists and cards
PATCH	/boards/:boardId	owner	Rename, edit description
DELETE	/boards/:boardId	owner	Delete board and its lists and cards

Members

Method	Path	Role	Purpose
GET	/boards/:boardId/members	member+	List members
POST	/boards/:boardId/members	owner	Invite by email (user must exist)
DELETE	/boards/:boardId/members/:userId	owner, or self	Remove member / leave

Lists

Method	Path	Role	Purpose
POST	/boards/:boardId/lists	owner	Create list
PATCH	/lists/:listId	owner	Rename or change position
DELETE	/lists/:listId	owner	Delete list and its cards

Cards

Method	Path	Role	Purpose
POST	/lists/:listId/cards	member+	Create card

Method	Path	Role	Purpose
GET	/cards/:cardId	member+	One card
PATCH	/cards/:cardId	member+	Edit fields
PATCH	/cards/:cardId/move	member+	Body: target list id and new position
DELETE	/cards/:cardId	member+	Delete card

Conventions

- Status codes: 201 for create, 204 for delete, 400 or 422 for validation, 401 not logged in or token expired, 403 logged in but wrong role, 404 not found, 409 duplicate (e.g. email already registered, user already a member).
- For a board the user does not belong to, prefer 404 over 403 so you do not reveal that the board exists.
- One consistent error shape, e.g. a message plus an optional field-errors object.
- Never trust ids in the body that should come from the URL or token (e.g. owner, createdBy).

7. Backend structure (suggested)

- routes: only wiring URLs to controllers
- controllers: read request, call logic, send response
- middleware: authenticate (verify access token, attach user), loadBoardMembership (find the board, find this user in members, attach role), requireOwner, validate, central error handler
- models: one file per model

Request flow for a card edit: authenticate, find card, load its board membership, confirm the user is a member, validate body, update, respond.

8. Frontend design

Pages: Login, Register, Boards dashboard, Board view.

State

- Auth context holding the user and access token in memory

- On app load, call refresh once to restore the session from the cookie
- An API client wrapper that, on a 401, calls refresh once, then retries the original request. If several requests fail at the same time, only one refresh should happen and the others wait for it. This is the trickiest frontend part of the project.

Drag and drop

- Library options: dnd-kit (recommended, actively maintained) or @hello-pangea/dnd (closest to the classic react-beautiful-dnd). Read the "sortable" and "multiple containers" sections of whichever you pick.
- Flow: drop event gives you source list, target list, and the neighbor cards, so compute the new position, update local state immediately (optimistic update), call the move endpoint, and roll back with an error message if it fails.

Role-aware UI: owners see list management, invite, and delete controls; members do not. This is only for convenience. The backend is the real gate.

9. Security and validation checklist

- Validate every request body (look at the docs for zod, joi, or express-validator and pick one)
- Rate limit login and register routes
- CORS limited to your frontend origin, with credentials enabled
- Do not log passwords or tokens
- Secrets in environment variables, never committed
- Check that the assignee on a card is a member of that board
- Escape or ignore unexpected fields so users cannot set role, owner, or password hash through a body

10. Build plan in micro steps

Each step has a **Look up** (where to read) and a **Checkpoint** (how you know it works). Test the backend with Postman, Thunder Client, or Insomnia before touching React, the same way you did on the notes app. Do not move to the next phase until the checkpoints pass.

Realistic goal for today: finish Phases 0 to 3. The whole project is more like 1 to 2 weeks of evenings.

Phase 0: Setup (about 30 min)

- [ ] Create a repo with two folders, one for the server and one for the client
  - Look up: Express "Hello world" and "Routing" in the Express guide
  - Checkpoint: a health route returns JSON in your API client
- [ ] Add environment variables for port, Mongo URI, and two separate JWT secrets (access and refresh)
  - Look up: the dotenv package README
  - Checkpoint: server logs the port from env and fails loudly if a secret is missing
- [ ] Connect to MongoDB Atlas the way you did in the expense tracker
  - Checkpoint: a "connected" log line, and the app exits with a clear error if connection fails
- [ ] Add a central error handler and a JSON body parser
  - Look up: Express guide, "Error handling"
  - Checkpoint: throwing inside a route returns a JSON error, not an HTML stack trace
- [ ] Add a git ignore for env files and node_modules, then commit
  - Checkpoint: your env file does not appear in the staged files

Phase 1: Register (about 1 hour)

- [ ] User model with the fields above, email unique and lowercase
  - Look up: Mongoose docs, Schemas, "Validation" and the unique index note
  - Checkpoint: saving two users with the same email fails
- [ ] Hash the password before saving
  - Look up: bcrypt package README (hash and compare)
  - Checkpoint: the database shows a long hash, never the plain password
- [ ] Register route with input validation
  - Checkpoint: bad email, short password, duplicate email each return a different clear 4xx; success returns 201 with no password hash in the response

Phase 2: Login and access token (about 1 hour)

- [ ] Login route: find by email, compare password, same generic error for either failure
  - Checkpoint: wrong email and wrong password return identical responses
- [ ] Sign an access token containing the user id, with a short expiry

  - Look up: jsonwebtoken package README (sign and verify), jwt.io introduction
  - Checkpoint: pasting the token into jwt.io shows your payload and an expiry time
- [ ] Authenticate middleware: read the Bearer header, verify, attach the user to the request
  - Checkpoint: a protected test route returns 401 with no token, 401 with a garbage token, 200 with a valid one
- [ ] Users/me route
  - Checkpoint: returns the logged-in user's profile without the hash

Phase 3: Refresh tokens (about 1.5 hours, take your time)

- [ ] RefreshToken model with the fields above and a TTL index
  - Look up: MongoDB docs, "Expire Data from Collections by Setting TTL"
  - Checkpoint: documents appear in the database after login, with a hash, not the raw token
- [ ] On login, create a refresh token, store its hash, send it as an httpOnly cookie
  - Look up: Express `res.cookie`, cookie-parser README, MDN Set-Cookie (HttpOnly, SameSite, Secure)
  - Checkpoint: your API client shows the cookie, and it is flagged HttpOnly
- [ ] Refresh route: read the cookie, hash it, find the stored record, check not expired and not revoked, issue a new pair, mark the old one replaced
  - Checkpoint: calling refresh twice with the same old cookie fails the second time
- [ ] Reuse detection: presenting a replaced token revokes all of that user's sessions
  - Look up: OWASP Session Management and JWT cheat sheets, section on refresh token rotation
  - Checkpoint: after a reuse attempt, even the newest refresh token stops working
- [ ] Logout and logout-all
  - Checkpoint: after logout, refresh fails and the cookie is cleared
- [ ] Tighten: rate limit the auth routes
  - Look up: express-rate-limit README
  - Checkpoint: the 6th rapid login attempt gets a 429

Phase 4: Boards and ownership (about 1.5 hours)

- [ ] Board model with owner and members (owner inside members with role owner)

- Checkpoint: creating a board stores you as owner and as the first member

- [ ] Create board, list my boards
  - Checkpoint: user A cannot see user B's boards in the list
- [ ] loadBoardMembership middleware that attaches the board and the caller's role, and returns 404 if the caller is not a member
  - Checkpoint: user B requesting user A's board gets 404
- [ ] requireOwner middleware
  - Checkpoint: a member gets 403 on rename and delete; owner succeeds
- [ ] Get one, rename, delete board (delete should also remove its lists and cards later; leave a note for yourself)

Phase 5: Members (about 1 hour)

- [ ] Invite by email: owner only, user must exist, no duplicates
  - Checkpoint: inviting twice returns 409; unknown email returns 404
- [ ] List members, remove member, member leaves
  - Checkpoint: a removed user immediately loses access to the board
- [ ] Block the owner from leaving
  - Checkpoint: the owner gets a clear error

Phase 6: Lists (about 1 hour)

- [ ] List model and create endpoint with auto position (last position plus a gap such as 1000)
  - Checkpoint: three created lists have increasing positions
- [ ] Rename, reposition, delete (delete cascades to cards once cards exist)
  - Checkpoint: a member gets 403 on all three
- [ ] Include lists, sorted by position, in the single-board response

Phase 7: Cards (about 1.5 hours)

- [ ] Card model, create endpoint, position at the end of the list
  - Checkpoint: card board field is set from the list, not from the request body
- [ ] Get one, edit fields, delete
  - Checkpoint: assigning a user who is not on the board is rejected
- [ ] Include cards, grouped per list and sorted by position, in the single-board response

- Checkpoint: one request returns everything the board page needs

Phase 8: Move and reorder (about 1.5 hours)

- [ ] Move endpoint: body has target list and new position
  - Look up: search "fractional indexing" and read one explanation
  - Checkpoint: moving between lists updates the list field and the position only
- [ ] Validate that the target list belongs to the same board
  - Checkpoint: moving a card to a list from another board is rejected
- [ ] Rebalance helper for when positions get too close
  - Checkpoint: after many moves into the same gap, positions are renumbered and the order is unchanged
- [ ] Finish cascades: deleting a list deletes its cards, deleting a board deletes lists and cards
  - Checkpoint: nothing orphaned in the database after a board delete

Phase 9: Frontend auth (about 2 hours)

- [ ] Create the React app, add a router, build Login and Register pages
  - Look up: React Router docs, "Tutorial" and "Protected routes" patterns
  - Checkpoint: registering and logging in from the UI works
- [ ] Auth context holding user and access token in memory
  - Look up: React docs, "Passing Data Deeply with Context"
  - Checkpoint: a page refresh loses the in-memory token, and that is expected for now
- [ ] Restore session on load by calling the refresh endpoint
  - Look up: your HTTP client's docs on sending cookies (axios `withCredentials` or fetch `credentials`)
  - Checkpoint: refreshing the page keeps you logged in
- [ ] API client wrapper that retries once after a 401 by refreshing
  - Look up: axios interceptors docs, or write a small fetch wrapper
  - Checkpoint: set the access expiry to 30 seconds temporarily; requests still succeed after waiting, with a refresh call in the network tab
- [ ] Protected route component
  - Checkpoint: logged-out users are redirected to login

Phase 10: Boards UI (about 2 hours)

- [ ] Boards dashboard: list, create, delete (owner only)
  - Checkpoint: boards you own and boards shared with you both show, with your role on each
- [ ] Board page: fetch the board and render lists and cards statically
  - Checkpoint: data matches what your API client shows
- [ ] Forms for add list, add card, edit card (modal or side panel)
  - Checkpoint: changes persist after a refresh
- [ ] Member panel: invite and remove
  - Checkpoint: the invited user sees the board on their dashboard

Phase 11: Drag and drop (about 3 hours)

- [ ] Pick one library and read its sortable and multiple-containers docs fully before coding
  - Checkpoint: you can explain what the source, target, and over/active items are in your own words
- [ ] Reorder cards inside one list (local state only)
  - Checkpoint: order changes visually, resets on refresh
- [ ] Move cards across lists (local state only)
  - Checkpoint: works with empty lists as drop targets too
- [ ] Compute the new position from neighboring cards and call the move endpoint
  - Checkpoint: order survives a refresh
- [ ] Optimistic update with rollback on failure
  - Checkpoint: stop the server, drag a card, see it snap back with an error message

Phase 12: Role-aware UI and polish (about 2 hours)

- [ ] Hide owner-only controls from members
  - Checkpoint: log in as a member; those buttons are gone, and calling the endpoints directly still returns 403
- [ ] Loading and error states, empty states
- [ ] Write a README with setup steps and screenshots
- [ ] Deploy (backend and frontend) and test cookies in production
  - Look up: your host's docs on cross-site cookies, since SameSite and Secure behave differently across domains

    - Checkpoint: login and refresh work on the deployed URL

Phase 13: Stretch (pick one at a time)

Labels, comments, activity log, list drag-and-drop, transfer ownership, pending invites. Each one needs a model change, routes, a permission decision, and UI. Write a mini spec for it first, like this document.

11. If you get stuck

- Write down what you expected, what happened, and the exact error text before searching.
- Reduce the problem: test the route in your API client in isolation before blaming the frontend.
- Read the relevant official docs section named above first, then search for the error.
- Commit after every checkpoint so you can always go back.